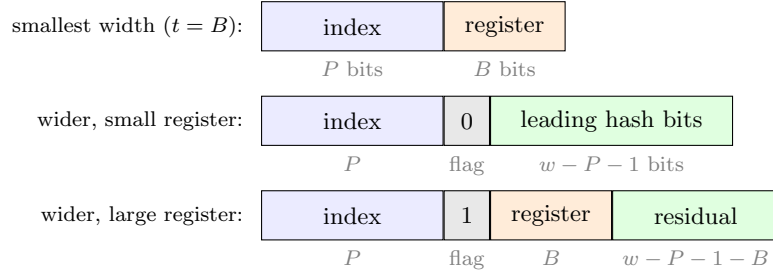


Hash-list cardinality correction (table-free occupancy inverse)

Setting. We want the number of distinct items n in a large stream while storing far less memory than the items themselves. This is cardinality estimation, and HyperLogLog (HLL) is the standard tool. It hashes each item and keeps an array of $m = 2^P$ small counters called registers, and from their distribution it estimates n with a relative error of about $1.04/\sqrt{m}$ regardless of n . That array is the dense representation. While the set is small this implementation is more accurate than registers, because it keeps the items explicitly in two compact stages before falling back to registers. First a value list, the exact sorted list of inserted values, so the count is exact. Then a hash list, a compact sorted list of the distinct hashes of the items. Only when the hash list no longer fits does the counter convert to dense registers. This note is about the estimator for the hash-list stage.

In the hash list each item is reduced to a composite hash, a w -bit fingerprint built from the register-bucket index (P bits), the item's leading-zero count, and a few residual bits. The list keeps the distinct composites sorted, and to stay compact it stores only the gap from each composite to the next, Rice-coded (a variable-length encoding cheap for small integers, with a per-cell optimal parameter). Sorting is what makes those gaps small. Their number D is what we observe. While w is large, distinct items almost never share a composite, so D equals the true count n . To pack more items as the list fills, the counter shrinks w , and then two distinct items can collide onto the same composite, so D starts to undercount n . The correction below recovers n from the observed D and the current width w .

What a hash-list entry is. A stored composite is a w -bit string. The top P bits are the register-bucket index, and the tail ($t = w - P$ bits) holds the register (the leading-zero count, what dense HLL keeps) plus extra resolution that is kept while w is wide. The tail takes three forms.



The index selects the bucket and the register is what dense HLL would store. The leading bits or residual are the extra resolution the counter drops as it shrinks w to pack more entries, which is what lets distinct items collide. The diagram is one composite's logical layout. On disk the entries are sorted and each is stored as the Rice-coded gap to the previous one, not in full.

Notation. Precision P (so $m = 2^P$ registers), bits per register B , current composite width w , tail $t = w - P$, observed distinct composites D , and largest storable register value $R = 2^B - 1$.

Write $g(n) := \mathbb{E}[D \mid n, w]$ for the expected distinct-composite count at cardinality n . The estimate $\hat{n}$ is the cardinality whose expected count matches the observed D , that is, g inverted at D (computed by safeguarded Newton iteration¹):

$$\mathbb{E}[D \mid \hat{n}, w] = D, \quad \text{equivalently} \quad \hat{n} = g^{-1}(D). \quad (1)$$

Equation (2) counts how many distinct composite values get used. Picture each possible composite as a box and each of the n items as landing in the box named by its composite. Two items in the same box collide, so the number of non-empty boxes (the distinct count D) falls below n , the same effect as shared birthdays in a room. The expected number of non-empty boxes, summed over groups of g_j equally likely

¹Newton's method solves $g(n) = D$ by repeatedly jumping to where the tangent of g meets D : from a guess it moves to $n - (g(n) - D)/g'(n)$. That is fast but can overshoot or run away when the curve is flat. The safeguard keeps an interval $[\text{low}, \text{high}]$ known to contain the answer (here $\text{low} = D$, with high doubled until $g(\text{high}) \geq D$) and takes the Newton jump only when it lands inside that interval, otherwise it halves the interval instead (bisection). Halving is slow but always shrinks the interval and never leaves it, so the method always reaches the answer.

boxes of probability p_j , is (2), and its derivative (3) drives the Newton step:

$$\mathbb{E}[D \mid n, w] = \sum_j \underbrace{g_j}_{\text{cells in group } j} \underbrace{\left(1 - (1 - p_j)^n\right)}_{\text{Pr[cell occupied by } n \text{ elements}]}, \quad (2)$$

$$\frac{\partial}{\partial n} \mathbb{E}[D \mid n, w] = \sum_j g_j \underbrace{(-\ln(1 - p_j))}_{>0} (1 - p_j)^n. \quad (3)$$

The cell groups follow from the composite encoding (uniform P -bit index, geometric register, uniform residual). At the smallest width $t = B$ the tail is just the register, giving (4). At wider widths a flag bit selects between storing the leading hash bits or the register plus a residual, giving (5):

$$\mathbb{E}[D \mid n] = 2^P \left[\underbrace{\sum_{r=1}^{R-1} (1 - (1 - 2^{-(P+r)})^n)}_{\text{geometric registers } r=1, \dots, R-1} + \underbrace{(1 - (1 - 2^{-(P+R-1)})^n)}_{\text{saturating register}} \right]. \quad (4)$$

$$\mathbb{E}[D \mid n, w] = 2^P \left[\underbrace{\sum_{\ell=0}^{\min(B, t-2)} 2^{t-2-\ell} (1 - (1 - 2^{-(w-1)})^n)}_{\text{flag 0: } r \leq B+1, \text{ leading bits stored}} + \underbrace{\sum_{r=B+2}^{R-1} 2^{t-1-B} (1 - (1 - 2^{-(P+r+t-1-B)})^n)}_{\text{flag 1: register+residual}} + \underbrace{2^{t-1-B} (1 - (1 - 2^{-(P+R-1+t-1-B)})^n)}_{\text{saturating register}} \right]. \quad (5)$$

The estimator in pseudocode. The uncorrected estimate is just the stored distinct count. The corrected estimate inverts (2) for n .

```

1: function UNCORRECTED
2:   return  $D$                                  $\triangleright D$  is the number of distinct composites stored
3: end function

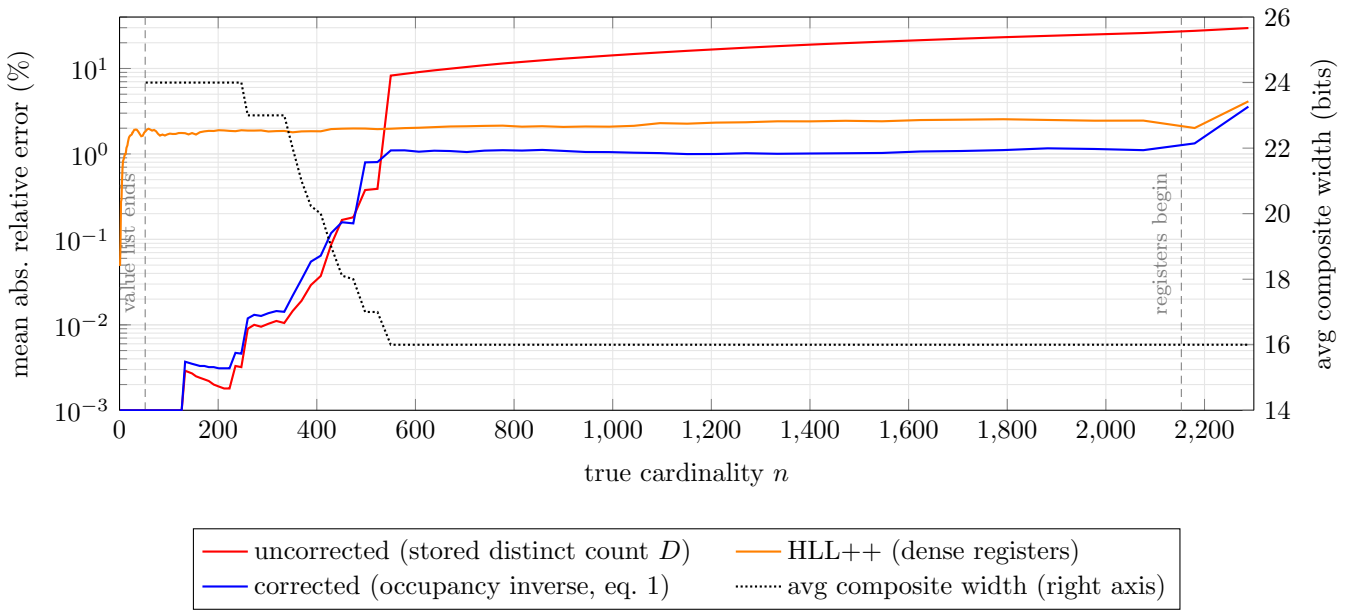
4: function CORRECTED( $D, w$ )                     $\triangleright w$  is the current width,  $P$  and  $B$  fixed
5:    $E(n) \leftarrow \sum_{(g,p)} g (1 - (1 - p)^n)$        $\triangleright$  cell groups  $(g, p)$  of eq. (4)/(5)
6:    $E'(n) \leftarrow \sum_{(g,p)} g (-\ln(1 - p))(1 - p)^n$ 
7:    $low \leftarrow D, \quad high \leftarrow D$ 
8:   while  $E(high) < D$  do                         $\triangleright$  grow until the root is bracketed
9:      $high \leftarrow 2 \cdot high$ 
10:  end while
11:   $n \leftarrow (low + high)/2$ 
12:  repeat
13:    if  $E(n) > D$  then
14:       $high \leftarrow n$ 
15:    else
16:       $low \leftarrow n$ 
17:    end if
18:     $candidate \leftarrow n - (E(n) - D)/E'(n)$        $\triangleright$  Newton step
19:    if  $low < candidate < high$  then
20:       $n_{\text{next}} \leftarrow candidate$ 
21:    else
22:       $n_{\text{next}} \leftarrow (low + high)/2$            $\triangleright$  bisection fallback
23:    end if
24:     $\delta \leftarrow |n_{\text{next}} - n|$ 
25:     $n \leftarrow n_{\text{next}}$ 
26:  until  $\delta \leq 10^{-12} n$                          $\triangleright$  relative step below  $10^{-12}$ 
27:  return  $n$ 
28: end function

```

Complexity. UNCORRECTED is $O(1)$: the distinct count D is maintained in the counter's metadata and read directly, not recomputed by scanning the stored entries. CORRECTED is $O(2^B)$ per estimate: E and

E' each sum over the $O(2^B)$ cell groups, and the safeguarded Newton loop runs a bounded number of iterations (plus $O(\log(n/D))$ steps to grow the bracket). It depends on neither the cardinality n nor the register count $m = 2^P$, and never scans the stored entries. Each cell group evaluates a logarithm and an exponential, so its constant is larger than the dense estimate’s $O(m)$ register scan, which is why the estimate-timing figure shows it slower in absolute terms despite touching fewer items. A hash-list insert, by contrast, splices into the sorted buffer and is $O(D)$ in the worst case, so its cost grows as the list fills, while a register insert is $O(1)$.

Accuracy at $P = 10$, $B = 6$: mean absolute relative error over 256 seeds (one seed cannot rank unbiased estimators, only the spread over seeds can). The uncorrected count D is biased low and worsens toward conversion. HLL++ (the dense estimate of the same elements) is unbiased but carries the full $1.04/\sqrt{m}$ register noise. The occupancy inverse (1) stays below both. The dotted curve (right axis) is the average composite width, which falls from 24 to 16 bits ($= P + B$) as the list fills, and each narrowing raises the collision rate that drives the uncorrected error up. Dashed verticals: value list ends at $n = 52$, registers begin near $n = 2153$.

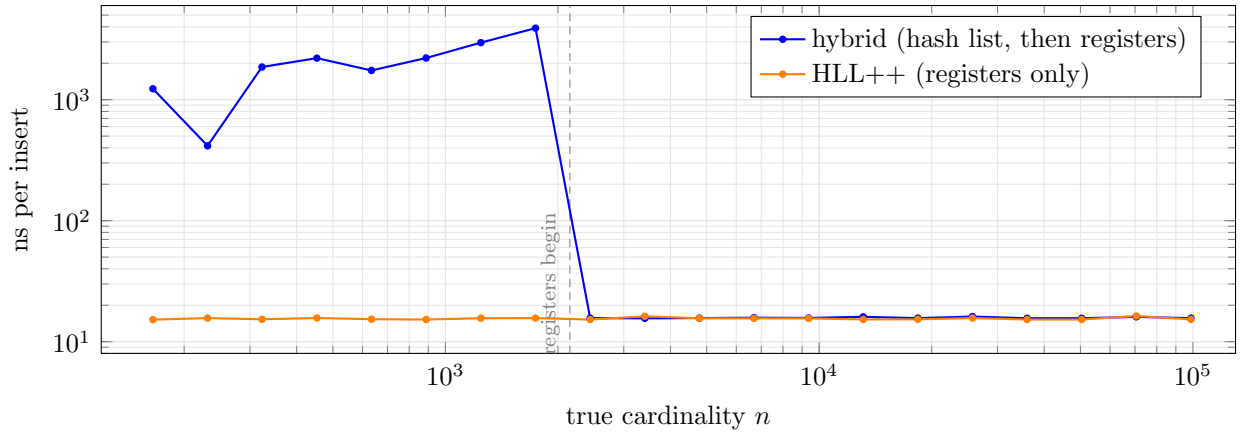


Aggregate errors over the hash-list range (256 seeds, $P = 10$, $B = 6$): MAE is the mean absolute error in elements, MRE the mean (signed) relative error, MSE the mean squared error in elements².

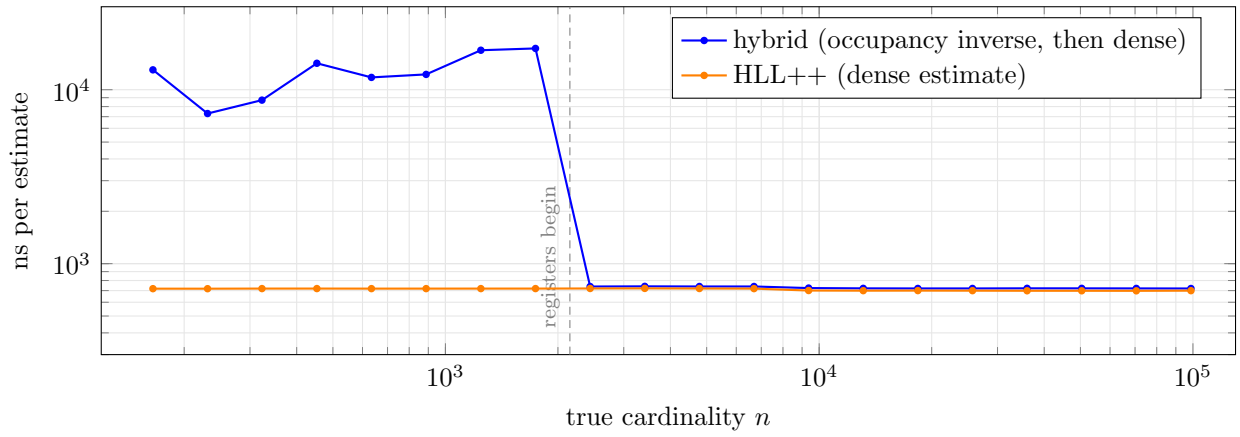
estimator	MAE (elements)	MRE	MSE (elements ²)
uncorrected (D)	208.9	−14.00%	76286
HLL++ (dense registers)	25.9	−0.01%	1398
corrected (occupancy inverse)	11.5	−0.05%	297

The corrected estimate is about $2.3\times$ better than HLL++ on MAE and $4.7\times$ on MSE, both are unbiased (MRE near zero), while the uncorrected count is biased low by 14%.

That accuracy is not free: the hybrid pays a much higher per-insert cost in the hash list than the dense registers. Mean per-insert time (200 reps, log-log): in the hash list each insert splices into a sorted gap-coded buffer at roughly 1 to 4 microseconds, about 80 to 250 times the ~ 16 ns register insert, until conversion near $n = 2153$ collapses it to register speed.



The same trade-off appears when estimating: the table-free occupancy inversion (a Newton solve over the cell groups) costs more per query than the dense estimate, about 7 to 17 microseconds in the hash list versus ~ 0.7 microseconds, again collapsing to dense cost after conversion. This is per query, not per element, so it matters far less than the insert cost.



Why no table. The correction is derived analytically from the encoding, under HyperLogLog’s usual uniform-hash assumption, so it stores nothing. A calibrated per-cell table fed the observed duplicate count would lower variance, but only by 5 to 18 percent in the tail, inside the $1.04/\sqrt{m}$ floor we already clear, and it costs about 38 KB of coefficients (roughly 1600 lines, 72 breakpoints per cell over 45 cells) regenerated by a multi-hour Monte Carlo per precision and bit-width set, plus insertion-order dependence. Table-free avoids all of it: nothing to fit or rerun, correct at every precision, never out of sync with the encoding, and path-independent.